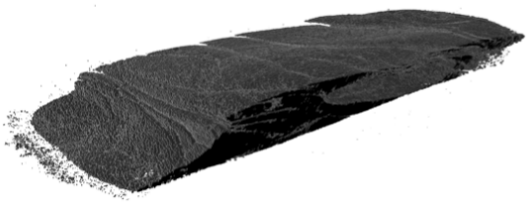
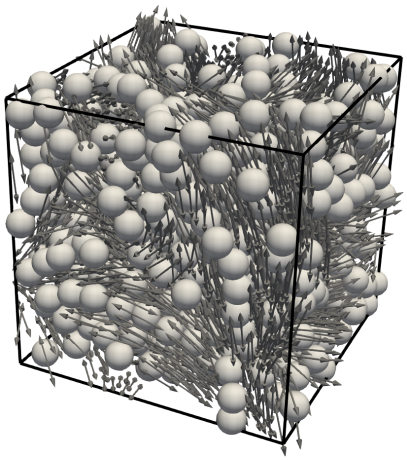
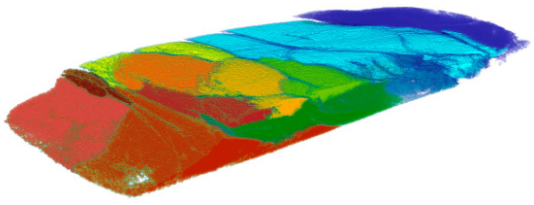
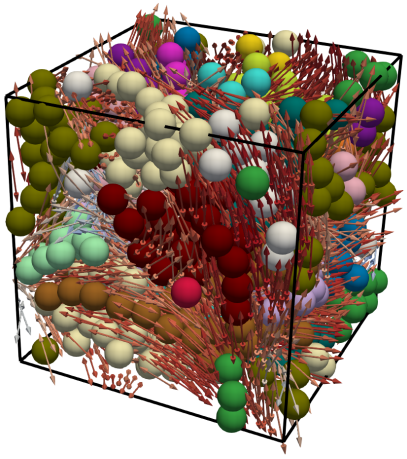
SEGVIZ TOOL USER GUIDE

Pauline Chassonnery*[1], Diane Peurichard†[2], and Sinan Haliyo‡[3]

[1]Bureau de Recherches Géologiques et Minières, Orléans, France
[2]Laboratoire Jacques-Louis Lions, Equipe MUSCLEES, UMR 7598, Sorbonne Université, Inria, CNRS, Université Paris Cité, Paris, France
[3]Institut des Systèmes Intelligents et de Robotique, UMR 7222, CNRS, Sorbonne Université, Paris, France

July 14, 2026

Data visualization using Paraview	
raw *in-vivo* data	raw *in-silico* data
	
segmented *in-vivo* data	segmented *in-silico* data
	
Data segmentation using Python	

*email: pauline.chassonnery@ens-paris-saclay.fr
†email: diane.a.peurichard@inria.fr
‡email: sinan.haliyo@sorbonne-universite.fr

Contents

1 About SegViz

The objective of the SegViz tool is to provide non-specialists in image processing with an integrated, user-friendly tool for the 3D visualization of systems composed of a mixture of spherical (e.g. cells, particles) and rod-like (e.g. fibers, bacteria) elements and for the automatic detection of spatially connected clusters of spherical objects separated by rod-like elements in such systems. It is aimed in particular at modelers, who may generate such an *in silico* dataset through numerical simulations, and at biologists who may retrieve it from *in vivo* or *in vitro* experiments (for example through 3D microscopy imaging of tissue samples).

Despite recent improvements in the field of high resolution tri-dimensional imaging techniques, the effective 3D visualization of the large datasets remains a challenge. This is particularly the case if a common platform is to be used for both biological images and mathematical data. Robust scientific visualization tools like the Visualization Toolkit (VTK) and its front-end application, Paraview [4], can bridge the gap between these two types of data. Yet, few efforts have been put in the development of plugins adapted to biological data and models.

On the other hand, objects segmentation and clustering is a wide-spread problem in the domain of image analysis and many methods have been developed to solve it, of which one of the most widely used is the watershed transformation [6]. But these are generally not well-known to researchers outside this field and the procedures provided by image analysis software usually require a number of preprocessing steps. Moreover, data produced by a mathematical simulation will usually not even be in the form of images and will thus require an extra processing step that can be quite time-consuming if not optimized.

The visualization part of the SegViz tool consists of two Paraview macros, **Sphereviz** and **Rodviz**, described in chapter 3. They take as input a .csv file describing the position, size and other optional properties of a set of spherical (resp. rod-like) objects and display them as 3D glyph that can be colored according to any property referenced in the dataset. By default, the spherical objects are colored according to their "ClusterIndex" if that information is available (see below) and white otherwise.

Considering that Paraview's preset discrete color maps are either limited to 12 colors or contain colors that are not easy to distinguish, we provide a custom categorical color map based on the work of Sasha Trubetskoy [5]. It contains 20 colors listed in descending order of compatibility with color blindness. This color map can be loaded in any Paraview setup and will be used by the macro **Sphereviz** if it is present. If not, the default Paraview color map "KAAMS" will be used.

The segmenting part of the SegViz tool consists in a Python function, called **WatershedSegmentation**, which is described in chapter 4. It takes as input either a binary image (in the form of a numpy.ndarray or .tiff file) or a list of spherical objects with

their properties (in the form of a pandas.DataFrame or .csv file), as well as a number of fine-tuning parameters. In the first case, it returns a labeled image where each region has been attributed a unique label. In the second case, it groups the objects into clusters based on spatial proximity and returns a copy of the pandas.DataFrame with an additional column containing the index of the cluster each object pertains to. In both cases, the segmentation step uses the watershed transformation [6].

Because conventional clustering software are made for biological or physical images, they never include tools to treat the case of periodic boundary conditions. However, this is a very common hypothesis in computational models that needs to be taken into account for clustering. The **WatershedSegmentation** function thus includes an option for periodic boundary conditions.

The SegViz tool module is free of use, upon proper citation of [2] and of this user manual. The zipped package containing all the necessary files is downloadable here.

2 Installation

2.1 Prerequisites

The SegViz tool requires Python 3.8 or higher, the Python packages numpy, pandas, Pillow and scikit-image, and Paraview 5.4 or higher.

Considering that Python is preinstalled on most computers, first check your eventual version of Python by typing the instruction `python3 --version` in your command-line interface (CLI), which is accessible by typing "cmd" in Windows search bar or via the Terminal app for MacOs. If this return nothing or a number below 3.8, download the latest Python version from the official website. You can check that the installation went fine by running the instruction `python3 --version` again. See the official download tutorial for more details.

Once this is done, install the required Python packages by typing the following instruction in your CLI :

```
pip3 install 'numpy>=1.23.5' 'pandas>=2.0.0' 'Pillow>=9.5.0' 'scikit-image>=0.20.0'
```

Alternatively, you can use your CLI to navigate to the SegViz folder and then type the instruction :

```
pip3 install -r requirements.txt.
```

The latest release of Paraview can be downloaded from the official website. The minimal requirement for the SegViz tool is Paraview 5.4, which itself requires Python 3.8 to be installed first. Note that the illustrations in this guide were created using Paraview 5.13.2.

Summary list

- Python >= 3.8
- Python packages :
  - numpy >= 1.23.5
  - pandas >= 2.0.0
  - Pillow >= 9.5.0
  - scikit-image >= 0.20.0
- Paraview >= 5.4

2.2 Loading a macro in Paraview

Paraview macros are stored individually in .py files.
To load a macro in your Paraview setup, open the application and go to the menu bar. There, click on the "Import new macro..." entry in the "Macros" drop-down menu (see Figure 1) and select the appropriate .py file in the folder browser. The macro will then be accessible in the "Macros" drop-down menu under the same name as its original file, plus a number if disambiguation is needed.
Note that the macro's file will be copied to Paraview's configuration folder, so that the macro will still be usable if you move or even delete the original file. You can remove a macro from your Paraview configuration using the "Delete..." entry (or, in newer versions of Paraview, the "Edit Macros" entry) in the "Macros" drop-down menu (see Figure 1).

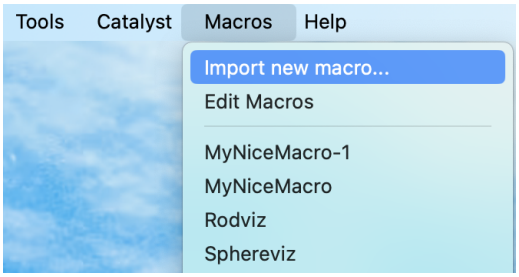


Figure 1: Screenshot of the "Macros" drop-down menu of Paraview menu bar.

The SegViz tool comprises two macros which can be installed independently, in any order. They are stored in the files `Sphereviz.py` and `Rodviz.py`.

2.3 Loading a color map in Paraview (advanced)

Paraview allows users to import custom color maps from json, xml or ct files. The SegViz tool comes with one such custom color map, based on the work of Sasha Trubetskoy [5] and stored in the file `SashaTrubetskoy.json`. It is used by the macro **Sphereviz** to make it easier to discriminate between different groups of objects in a dataset.
Note that loading this color map isn't necessary for the macro to work (it will default to one of Paraview preset color map), so this somewhat tedious step of the installation can be safely ignored.

Loading a color map requires your Paraview window to be currently using one of its pre-installed color map to render a visual. Follows the instructions in section 3.2 below to display the dataset stored in file `demo_data/dataset_spheres_seg.csv` using the macro **Sphereviz**.

Once the rendering is done, open the Color Map Editor using the "View > Color Map Editor" entry in the menu bar. Depending on your settings, the editor may open in a side panel or as an additional window.
From the Color Map Editor, open the Presets dialog window by clicking on the "Choose Preset" button (see Figure 2), then click on the "Import" button (see Figure 3) and select the desired file using the folder browser.

Depending on your settings, the new map may not show up immediately, because the current color maps in view are only the default ones. If this is the case, you can change the parameter "Default" to "All" in the Presets window to reveal the new map : it will appear towards the end of the list, with its name in italics. You can select it and tick the "Show this map in Default" box to have it shown by default.
You can test the new color map by manually applying it to your current visual. Afterward, the macro **Sphereviz** will automatically use this map.

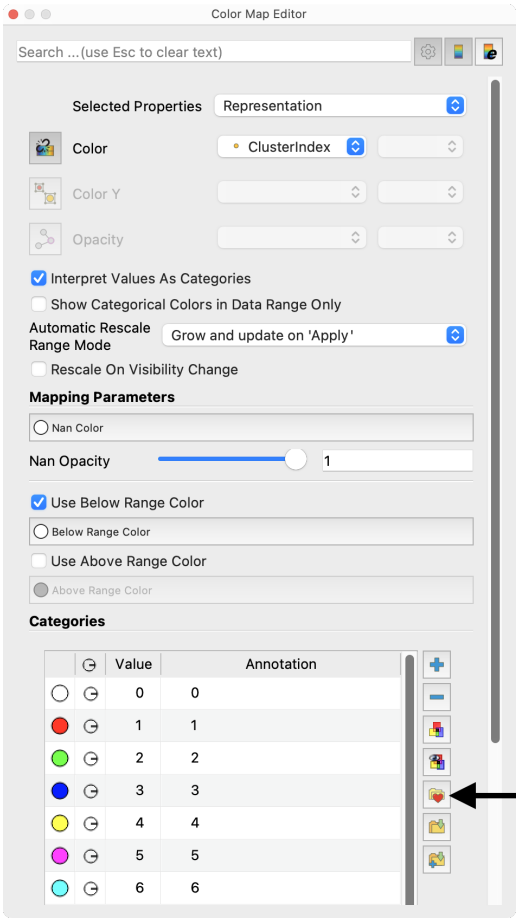


Figure 2: Screenshot of the Color Map Editor window, with a black arrow pointing the "Choose Preset" button.

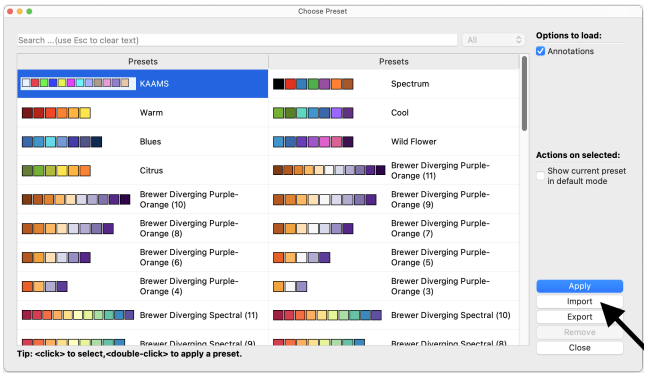


Figure 3: Screenshot of the Choose Preset window, with a black arrow pointing the "Import" button.

2.4 Getting started with python

A python script `main.py` can be created using any text editor and then run (that is, executed) from the command-line interface with the instruction `python3 main.py`. This is the most basic use and the less beginner-friendly.

The alternative is to use an Integrated Development Environment, that is an app which includes all the functionalities of a text editor, an integrated command-line interface (so that you don't have to switch apps to edit/run your program), a debugger and many programming assistance features such as syntax highlighting, formatting, code completion, etc.

A minimalistic Python IDE usually recommended for beginners is Thonny. It is free and open-source but very limited. For more complex uses, prefer Spyder (free, open-source, specifically designed for scientific purposes but not adapted to web development) or Visual Studio Code (free, widely used, supporting multi-languages but requiring specific configuration) and its Python extension. For a web-based interactive development environment, Jupyter Notebook (or the more recent JupyterLab) is commonly used.

Beginner's guides to Python are listed on the official wiki. We will only repeat here the points required to use the SegViz

tool.

To use in a script the content of another file (written by you or a third-party), you have to *import* it. In our case, the function **WatershedSegmentation** stored in the file `SegTool.py` can be imported using the following statement, usually at the beginnning of your script :

```
from SegTool import WatershedSegmentation
```

Note that the imported file must be located in the same folder than (or in a subfolder of) the script where it is imported.

3 Visualizing data

3.1 Visualizing 3D images stored in .tiff files (default use of Paraview)

Open data file

To open a 3D image contained in a .tiff (or .tif) file with Paraview, you can either :

(i) right-click on the file and select the action "Open with > Paraview";

(ii) open a Paraview window and use the "File > Open..." menu in the menu bar;

(iii) open a Paraview window and drag-and-drop the file to open inside the window.

In the first case, Paraview will open a popup window asking you to select the appropriate reader to use for this file (see Figure 4). Please select "TIFF Series Reader".

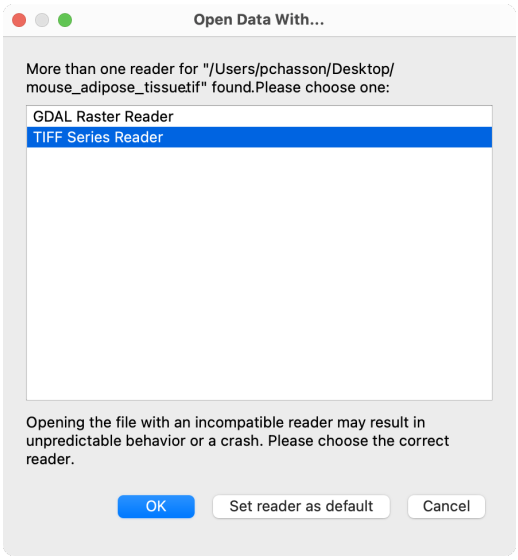


Figure 4: Screenshot of the "Open Data With..." popup window opened by Paraview for .tiff files.

Display data

The file should now appear in the "Pipeline Browser" panel of the Paraview window, and its various modifiable properties be displayed in the "Properties" panel (see Figure 5). Click on the "Apply" button at the top of the "Properties" panel to create the display. Alternatively, if the pixels of your image are not cubics, e.g. because the imaging setup does not have the same resolution in the z-direction than in the x and y ones, you can tick the "Use Custom Data Spacing" in the same panel and indicate the relative side-lengths of the pixels in the text boxes below before clicking on "Apply". See Figure 5 for an example with the file `mouse_adipose_tissue.tif` (stored in a compressed archive (Zip)) of the folder `demo_data`, whose pixels are 18.4µm ×18.4µm ×10µm wide.

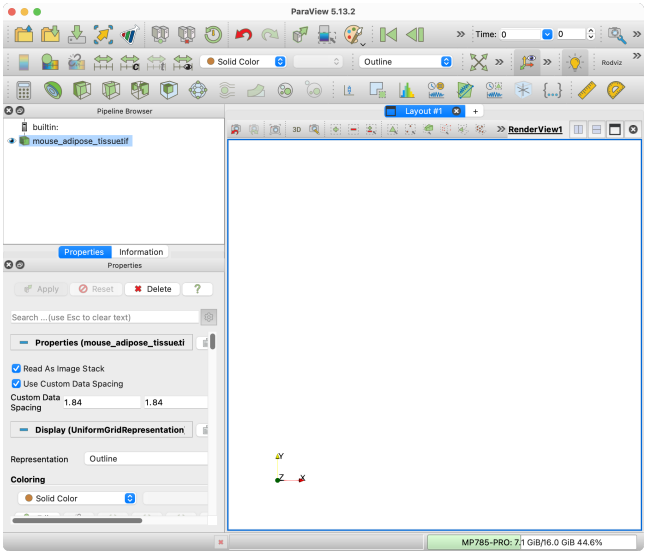


Figure 5: Screenshot of Paraview main window after loading the file `demo_data/mouse_adipose_tissue.tif` and setting custom data spacing.

Change default display style

By default, Paraview use the outline representation, which does not display anything if the content of the file is a binary or labeled image. Use the "Representation" drop-down menu which now appears in the "Properties" panel to select a more appropriate display, for example "Volume" (Figure 6).

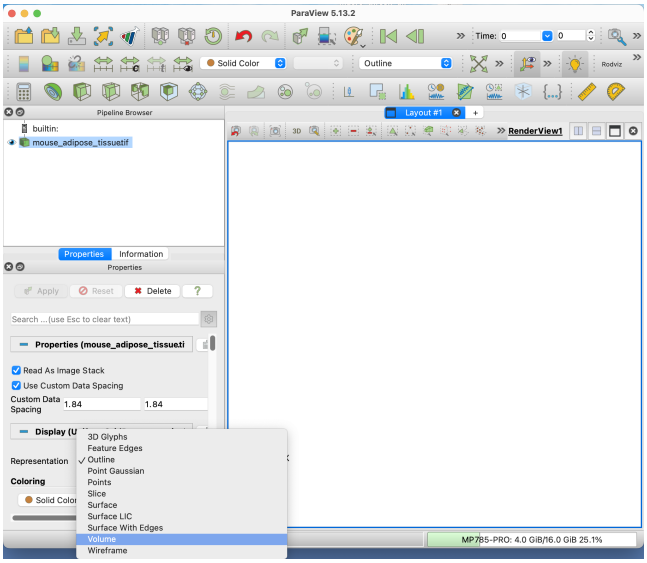


Figure 6: Screenshot of Paraview main window with open "Representation" drop-down menu in the "Properties" panel.

The data should now appear in the "Layout" panel of the Paraview window (see Figure 7). The view angle can be manipulated with the mouse. The default color map can be changed using the Color Map Editor (accessible via the "View > Color Map Editor" entry in the menu bar or the "Edit" button in the "Coloring" section of the "Properties" panel). For a labeled image, a well contrasted color map like the Rainbow Desaturated preset may be more appropriate.

Note that Paraview can not apply categorical (or discrete) color maps to .tiff images. A workaround is to apply to the image a "Threshold" filter with lower threshold 1 (to ignore background) and upper threshold equal to the maximum value in the image (automatically identified by Paraview), then render the thresholded data instead of the original.

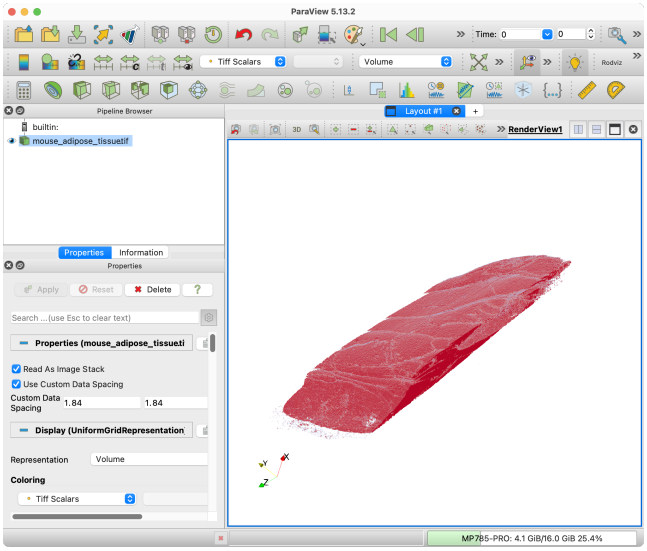


Figure 7: Screenshot of Paraview main window with the image `demo_data/mouse_adipose_tissue.tif` displayed in volume using default color map.

3.2 Visualizing datasets stored in .csv files

Open data file

To open a dataset contained in a .csv file with Paraview, you can either :

(i) right-click on the file and select the action “Open with > Paraview”;

(ii) open a Paraview window and use the “File > Open...” menu in the menu bar;

(iii) open a Paraview window and drag-and-drop the file to open inside the window.

In any case, Paraview will open a popup window asking you to select the appropriate reader to use for this file (see Figure 8). Please select “CSV Reader”.

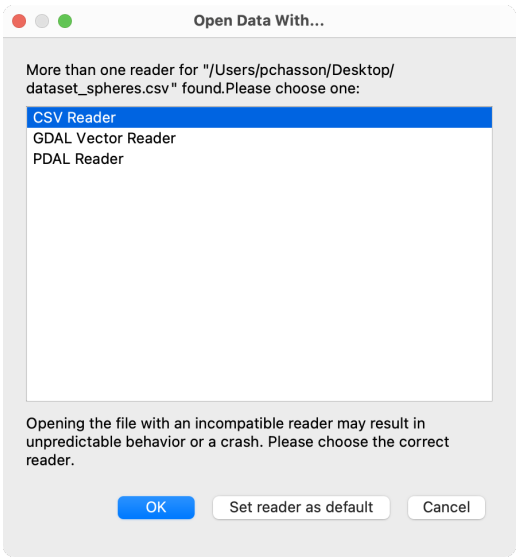


Figure 8: Screenshot of the “Open Data With...” popup window opened by Paraview for .csv files.

The file should now appear in the “Pipeline Browser” panel of the Paraview window, and its various properties be displayed in the “Properties” panel (see Figure 9). Click on the “Apply” button at the top of the “Properties” panel to load the dataset into a “SpreadSheetView” panel (you can close the corresponding panel to gain some space, without losing the data).

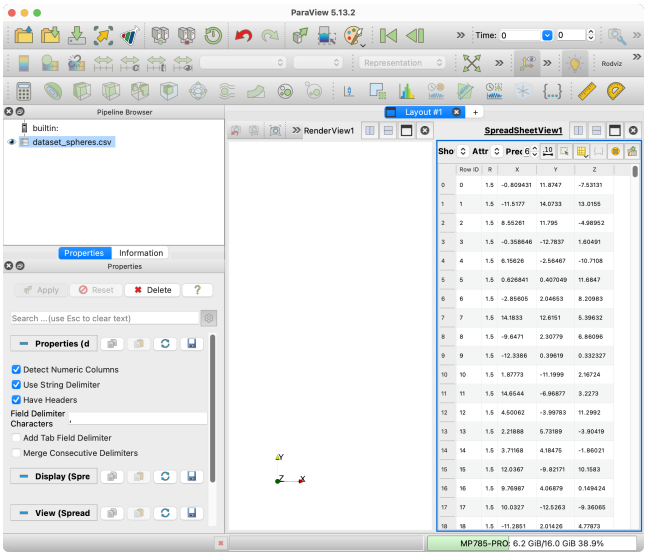


Figure 9: Screenshot of Paraview main window after loading the file `demo_data/dataset_spheres.csv` and clicking on the “Apply” button.

Apply a macro to a dataset

To apply a macro to a dataset, first select this dataset by clicking on its name in the “Pipeline Browser” panel. If the macro is expected to produce a visual, also make sure that the active visualization panel (outlined in blue) is the one where you want this visual to appear. You can activate the desired panel by clicking on it.

You can now apply the desired macro by clicking on its name in the “Macros” drop-down menu of Paraview menu bar (see Figure 1).

Apply macro Sphereviz

The macro **Sphereviz** take as input a dataset containing at least the four following properties (formatted as column header in the first line of the .csv file) : “X”, “Y”, “Z” and “R” pointing respectively the x-, y- and z-coordinate of the object center and its radius.

If the dataset also contains a property “ClusterIndex”, each glyph will be colored according to this property using the custom categorical (i.e. discrete) colormap “sashatrubetskoy” (if it is loaded in the Paraview setup) or the default categorical colormap “KAAMS”. Otherwise all the glyphs will be white.

Applying the macro **Sphereviz** to the dataset `demo_data/dataset_spheres.csv`, which does not contain a “ClusterIndex” column, will produce the visual displayed in Figure 10.A.

Applying it to the dataset `demo_data/dataset_spheres_seg.csv` will produce the visual displayed in Figure 10.B. This second dataset has been segmented assuming the spatial domain had periodic boundaries (see corresponding paragraph in section 4.3.2 below for an explanation). In order to better visualize the shape of the resulting clusters, one can use the translated coordinates of the objects instead of their original coordinates to achieve the visual displayed in Figure 10.C (see section 4.3.2 for more informations about these coordinates). To do this, first click on the “Spheres DataTable” item in the “Pipeline Browser” panel. Then, in the “Properties” panel, change the value of the “X Column”, “Y Column” and “Z Column” properties respectively from X, Y and Z to X_cluster, Y_cluster and Z_cluster (see Figure 10.D).

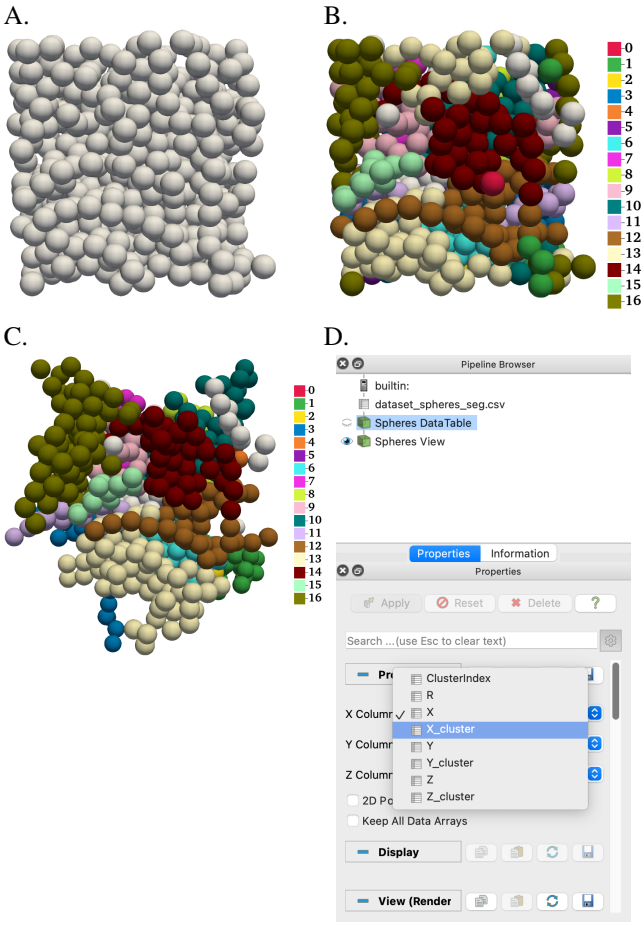


Figure 10: **A.** Screenshot of the visual generated by the macro **Sphereviz** applied to the dataset `demo_data/dataset_spheres.csv`. **B and C.** Screenshots of the visuals generated by the macro **Sphereviz** applied to the dataset `demo_data/dataset_spheres_seg.csv` using either the objects original coordinates (B) or their translated coordinates (C). **D.** Screenshot of Paraview “Properties” panel with open “X Column” drop-down menu.

Apply macro Rodviz

The macro **Rodviz** take as input a dataset containing at least the seven following properties (formatted as column header in the first line of the .csv file) : “X”, “Y”, “Z”, “wX”, “wY”, “wZ”, “L” and “R” pointing respectively the x-, y- and z-coordinate of the object’s center, the x-, y- and z-component of its unitary directional vector, its length and its radius.
If the dataset also contains a property “color”, each glyph will be colored according to this property using the default “CoolToWarm” colormap. Otherwise all the glyph will have a dark-grey color.

Applying the macro **Rodviz** to the dataset `demo_data/dataset_rods.csv`, which does not contain a “color” column, will produce the display presented in Figure 11.A.

Applying it to the dataset `demo_data/dataset_rods_colored.csv` will produce the display presented in Figure 11.B. To color the objects using another property of the dataset, select the wanted property in the list-menu of the “Coloring” section in the “Properties” panel.

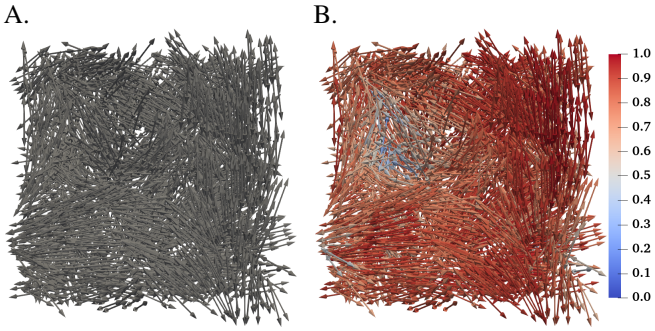


Figure 11: **A.** Screenshot of the visual generated by the macro **Rodviz** applied to the dataset `demo_data/dataset_rods.csv`. **B.** Screenshot of the visual generated by the macro **Rodviz** applied to the dataset `demo_data/dataset_rods_colored.csv`

4 Segmenting data

Our segmentation tool consists mainly in a Python function called **WatershedSegmentation** and stored in the script `SegTool.py`. This function takes two mandatory parameters - the first describes the type of data to treat (either a 3D binary image or a dataset of spherical objects), the second is the data in question - and saves the segmented data to a new file. The basic usage for both types of input is described in sections 4.1 and 4.2 below.

This function also has several optional parameters allowing the user to control the output format and to fine-tune the segmentation algorithm. All these parameters are listed in table 1 and their use is detailed in section 4.3.

The examples provided in this chapter are illustrated using our visualization tool (see chapter 3 above). Considering the difficulty of apprehending 3D data, this examples are based on simplified dummy data generating simple visuals. Examples based on real data are provided in the next chapter 5.

4.1 Segmenting 3D images stored in .tiff files

To segment a 3D image stored in a .tiff file (for example the file `bananas.tiff` from the folder `demo_data`, whose content is displayed in Figure 12.A), run Script 1. This will create a file named `data_seg.tiff` containing a labeled image where each pixel contains the index of the cluster to which it pertains. The indexes are continuous starting from 1, with 0 as background. The result is displayed in Figure 12.B, with the bounding box drawn in black and a transparent background for better readability.

The first parameter of the function **WatershedSegmentation**, named `datatype`, indicates the type of data to segment. The second, named `InputSpheres`, is the path to the file (using the format `folder/subfolder/filename`). Alternatively, if the image has already been read in the script (e.g. because you wanted to pre-process it), the resulting Python object can be used instead.

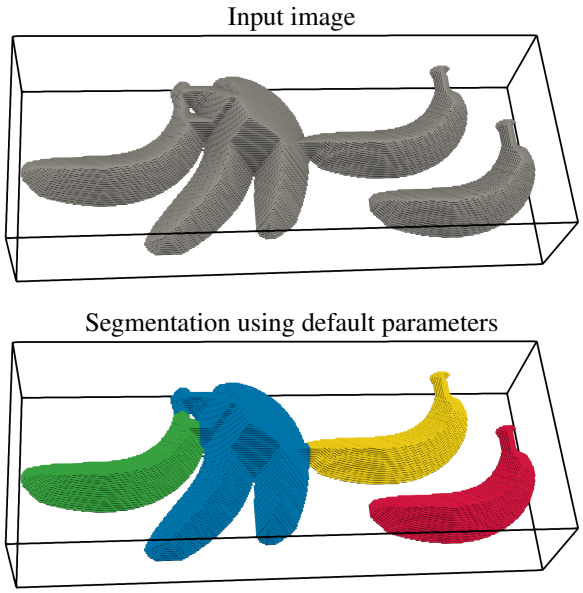


Figure 12: Segmentation of a 3D image using default parameters.

Python Script 1: Segmentation of a 3D image using default parameters.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("image", "demo_data/bananas.tiff")
```

More generally, the parameter `InputSpheres` must be a 3D binary-convertible image or a path to such an image, using one of the following formats :

(i) A *numpy.ndarray* of dimension 3 and boolean type.

(ii) A *numpy.ndarray* of dimension 3 and scalar real type (i.e. with `dtype` being a subtype "np.integer" or "np.floating"). The content of this array will be converted to boolean using the function **astype('bool')** and a warning will be issued to the user.

(iii) A *string* with the path to a .tiff (or .tif) file containing a 3D binary or grayscale image, which will be read to a *numpy.ndarray* using the functions **PIL.Image.open** and **PIL.ImageSequence.Iterator**. If the image is grayscale, it will be converted to binary using the function **astype('bool')** on the resulting array. The folder `demo_data` contains two examples of such a file, named `bananas.tiff` and `mouse_adipose_tissue.tif` (the latter stored in a compressed archive (Zip)).

4.2 Segmenting datasets stored in .csv files

To segment a dataset of spherical objects stored in a .csv file (for example the file `default.csv` from the folder `demo_data`, whose content is displayed in Figure 13.A using the Paraview macro **Sphereviz**), run Script 2. This will create a file named `data_seg.csv` containing the original dataset plus a column labeled "ClusterIndex" and holding the index of the cluster to which each object pertains (numbered continuously starting from 0, to be consistent with Python indexing). The result is displayed in Figure 13.B using the Paraview macro **Sphereviz**.

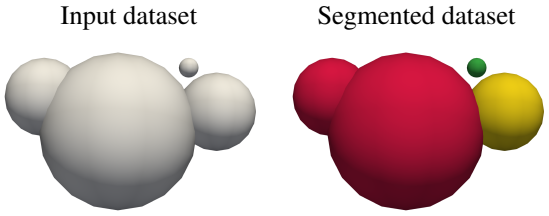


Figure 13: Segmentation of a dataset using default parameters.

Python Script 2: Segmentation of a dataset using default parameters.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/default.csv")
```

The first parameter of the function **WatershedSegmentation**, named `datatype`, indicates the type of data to segment. The second, named `InputSpheres`, is the path to the file (using the format `folder/subfolder/filename`). Alternatively, if the dataset has already been read by the script (e.g. because it was created by another part of the program), the resulting Python object can be used instead.

More generally, the parameter `InputSpheres` must be a dataset of spherical objects or a path to such a dataset, using one of the two following formats :

(i) A *pandas.DataFrame* with at least two rows (i.e. describing at least two objects, otherwise there is nothing to clusterize) and at least four columns with headers "X", "Y", "Z" and "R" containing respectively the x-, y- and z-coordinate of the object's center and its radius. The dataset may contain any number of additional columns.

(ii) A *string* with the path to a .csv file containing the data, which will be read to a *pandas.DataFrame* using the function **pandas.read_csv**. The file must use the default csv format, that is comma ',' for column separator and dot '.' for decimal point. The first line must contain column headers among which "X", "Y", "Z" and "R" pointing respectively the x-, y- and z-coordinate of the objects center and their radius. The order of the columns does not matter and the file may contain any number of additional columns. The folder `demo_data` contains a few examples of such files, among which `dataset_spheres.csv`.

4.3 Going further : optional parameters

The following sections describe all the optional parameters of the function **WatershedSegmentation**, summarized in Table 1.

4.3.1 Output settings

If the `datatype` is "image", the function **WatershedSegmentation** returns a .tiff file (named `savefile`.tiff) containing a labeled image where each pixel contains the index of the cluster to which it pertains. The indexes are continuous starting from 1, with 0 as background.

If the `datatype` is "dataset", the function **WatershedSegmentation** returns a .csv file (named `savefile`.csv) containing the original set of spherical objects plus the result of the clusterization process : a column labeled `header` for the cluster index and, if `PeriodicBoundaries` is True, three columns labeled "X`header_per`", "Y`header_per`" and "Z`header_per`" for the translated coordinates of the objects. In addition, if `return_map` is True, the function will returns a .tiff file (named `savefile`.tiff) containing the segmentation map.

`savefile` : name of the output file(s)

The parameter `savefile` enables the user to specify the name, without extension, of the above-mentioned output file(s).

Python type is *string*, default value is "data_seg".

`return_map` : whether to return the segmentation map

If the `datatype` is "dataset", the parameter `return_map` enables the user to ask for the segmentation map to be saved to as a labeled image in a .tiff file, in addition to the .csv file containing the segmented dataset. The name of both files will be taken from the parameter `savefile`.

Python type is *bool*, default value is False.

Name	Description	Python type	Default value	Used if datatype is "image"	Used if datatype is "dataset"
`savefile`	Name of the output file(s).	*string*	"data_seg"	x	x
`return_map`	Whether to return the segmentation map.	*bool*	False		x
`header`	Header of the column containing the clusterization result in the output file.	*string*	"ClusterIndex"		x
`header_per`	Suffix for the header of the three columns containing the translated coordinates in the output file.	*string*	"_cluster"		x
`InputRods`	Set of rod-like objects acting as cluster separators.	*string* or *numpy.ndarray* or *pandas.DataFrame*	None	x	x
`resolution`	Image resolution, in pixel per radius of the smallest object.	*int*	10		x
`xmax`, `ymax`, `zmax`	Half-length of the data spacial domain in each direction.	*float*	None		x
`Periodic Boundary Condition`	Whether the borders of the computational domain are considered periodic.	*bool*	False		x
`dil_coeff`	Scaling applied to the spherical objects before segmentation.	*float*	1.0		x
`pixelsize`	Length of the pixels in the three directions.	*list* of three *floats*	$[1.0, 1.0, 1.0]$	x	
`smooth_coeff`	Smoothing applied to the distance map before segmentation.	*float*	1.0	x	x
`MinSize`	Minimal size of a valid cluster (expressed in pixels if datatype is "image" and in number of objects if datatype is "dataset").	*int*	1	x	x

Table 1: Summary of the optional parameters of the function **WatershedSegmentation**.

`header`, `header_per` : labels for the segmentation result in the output file

If the `datatype` is "dataset", the parameters `header` and `header_per` enable the user to specify the label of the column(s) containing the result of the segmentation process in the output .csv file.

Parameter `header` is used for the column containing the cluster index of each object. If `PeriodicBoundaries` is True, the output .csv file will also contain the translated coordinates of the objects (see corresponding paragraph in section below). These three columns will be labeled respectively "X"+`header_per`, "Y"+`header_per` and "Z"+`header_per`.

Python type is *string*, default values are "ClusterIndex" and "_cluster".

4.3.2 Pre-processing : creating an image from a dataset

`InputRods` : rod-like cluster separators

The function **WatershedSegmentation** may be provided, via the parameter `InputRods`, with cluster separators to help the segmentation process.

If the `datatype` is "image" then `InputRods` must be a binary image or a path to such an image, using one of formats described in section 4.1 above. This image will be used as a negative mask applied to the image provided via `InputSpheres` before the actual segmentation process.

Figure 14.A takes up the example from section 4.1 (semi-transparent for better readability) and superimposes the image `bananas_separators.tiff` from the folder `demo_data` on top of it. Using the latter as a mask when segmenting the former produces 7 differents regions instead of the 4 initially identified. The result of this segmentation is displayed in Figure 14.B and can be reproduced by running Script 3.

Python type is then either *string* or *numpy.ndarray*, default value is None.

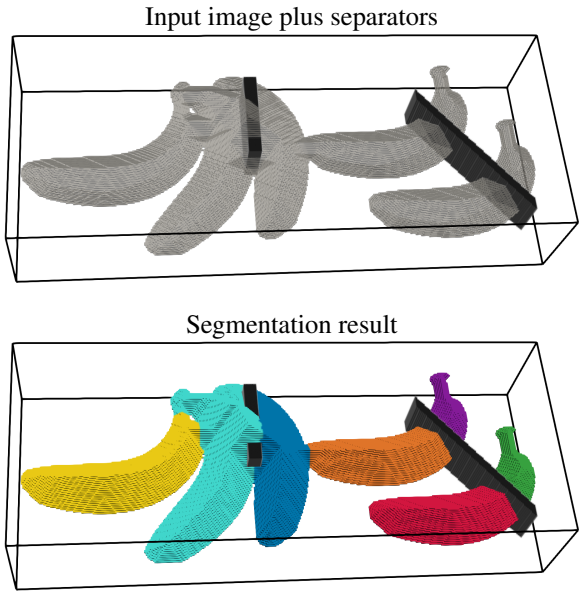


Figure 14: Segmentation of an image using another image as a mask to separate regions.

Python Script 3: Segmentation of a 3D image using a separator mask.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("image", "demo_data/bananas.tiff", InputRods="demo_data/bananas_separators.tiff")
```

If the `datatype` is "dataset" then `InputRods` must be a set of rod-like (or spherocylindrical) objects. These objects will be used as masks during the creation the binary image : that is, pixels located inside a rod-like object will be considered as background even if they are also inside a spherical object.

Taking up the example in section 4.2, the file `default_separators.csv` rom the folder `demo_data` can be added in to separate the two elements on the left-hand side. The datasets can be visualized together using the Paraview macros **Spherevizz** and **Rodviz** (see Figure 15.A). The result of the segmentation is displayed in Figure 15.B and can be reproduced by running Script 4.

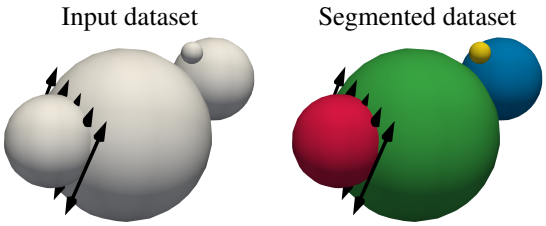


Figure 15: Segmentation of a dataset using rod-like separators (the view angle is tilted compared to Figure 13).

The dataset `InputRods` must be in one of the two following formats :

(i) A *pandas.DataFrame* with at least eight columns with headers “X”, “Y”, “Z”, “wX”, “wY”, “wZ”, “L” and “R” containing respectively the x-, y- and z-coordinates of the object center, the x-, y- and z-components of its unitary directional vector, its length and its radius. The dataset may contain any number of additional columns.

(ii) A *string* with the path to a .csv file containing the data, which will be read to a *pandas.DataFrame* using the function **pandas.read_csv**. The file must use the default csv format, that is comma ‘,’ for column separator and dot ‘.’ for decimal point. The first line must contain column headers, among which “X”, “Y”, “Z”, “wX”, “wY”, “wZ”, “L” and “R” pointing respectively the x-, y- and z-coordinate of the object’s center, the x-, y- and z-component of its unitary directional vector, its length and its radius. The order of the columns does not matter and the file may contain any number of additional columns. The folder `demo_data` contains two example of such a file, named `default_separators.csv` and `dataset_rods.csv`.

Default value is None.

Python Script 4: Segmentation of a dataset using rod-like cluster separators.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/default.csv", InputRods="demo_data/default_separators.csv")
```

`resolution` : number of pixels in the diameter of the smallest object

If the `datatype` is "dataset", the parameter `resolution` indicates the number of pixels that the radius of the smallest object in the input data will cover in the 3D binary image created for segmentation. In other words, the binary image will be composed of cubic pixels of side-length equal to the radius of this smallest spherical object divided by `resolution`.

The minimal value is 1, which ensures that all objects are represented by at least one pixel in the binary image. Setting `resolution` to a high value ensures that small objects are well outlined, but increases the total size of the image and thus the computational cost. Note that, from the perspective of cluster segmentation, a high resolution is usually necessary only if the clusters are very close to each other. An example of such a case is displayed in Figure 16. The results in this figure can be reproduced by running Script 5 and applying the Paraview macro **Spherevіz** (see section 3.2) to the files `resolution1.csv` and `resolution5.csv` thus produced.

Python type is *int*, default value is 5.

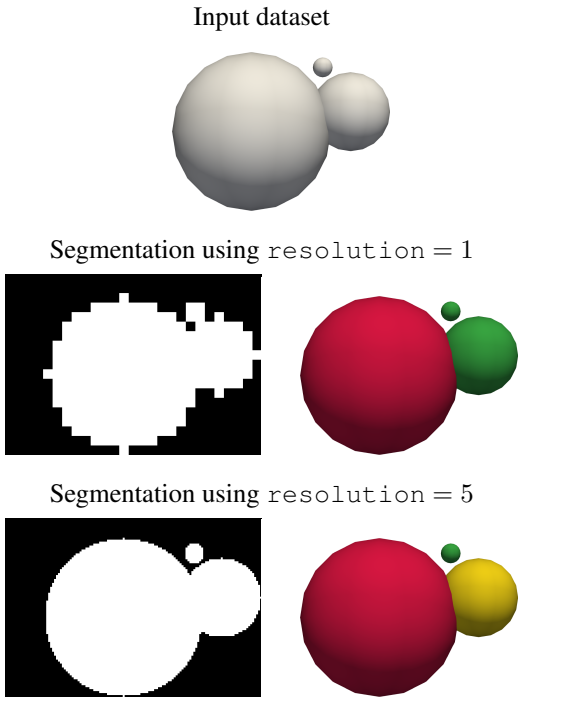


Figure 16: Illustration of the influence of the parameter `resolution` over the segmentation result. **Left column :** One slice of the 3D binary image created by the segmentation tool (objects appear in white and backgound in black). **Right column :** 3D visualization of the segmented dataset using Paraview (all objects pertaining to the same cluster have the same color).

Python Script 5: Segmentation of a dataset using different values of the parameter `resolution`.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/resolution.csv", savefile="resolution1", resolution=1)
WatershedSegmentation("dataset", "demo_data/resolution.csv", savefile="resolution5", resolution=5)
```

`xmax`, `ymax`, `zmax` : side-length of the computation domain

If the `datatype` is "dataset", the algorithm assumes the input data to be contained in a 3D symmetrical domain $[-\texttt{xmax}, \texttt{xmax}] \times [-\texttt{ymax}, \texttt{ymax}] \times [-\texttt{zmax}, \texttt{zmax}]$ and creates a 3D binary image spanning this whole domain.

If the user did not provide a value for one of the half-length parameters `xmax`, `ymax` and `zmax`, the algorithm will estimate the missing value(s) as the smallest value(s) allowing the domain to enclose all the objects referenced in the dataset `InputSpheres` :

$$\begin{aligned}\texttt{xmax} &= \max(|X_i| + R_i, \;\; i \in \texttt{InputSpheres})\\ \texttt{ymax} &= \max(|Y_i| + R_i, \;\; i \in \texttt{InputSpheres})\\ \texttt{zmax} &= \max(|Z_i| + R_i, \;\; i \in \texttt{InputSpheres})\end{aligned}$$

Figure 17 illustrates this process. Note that user-provided values are really needed only if `PeriodicBoundaries` is True, to find out which part of the objects cross the border of the domain (see next paragraph).

Python types are *float*. Default values are None, leading to automatic computation.

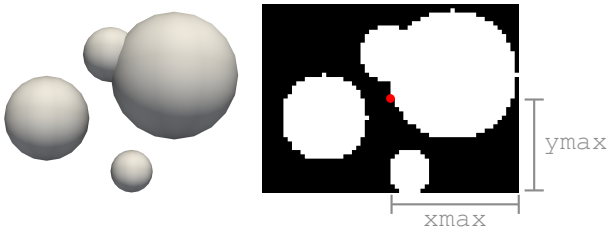


Figure 17: Example of automatic computation of the domain size from the input data. **Left :** 3D visualization of the input dataset using Paraview. **Right :** One slice (along the z-axis) of the 3D binary image created by the segmentation tool (objects appear in white and background in black). The center point $(0, 0)$ is marker by a red dot and the half-lengths `xmax` and `ymax` are indicated with gray lines.

`PeriodicBoundaries` : whether the input dataset assume periodic boundaries

If the `datatype` is "dataset" and this data is derived from a mathematical model including periodic boundary conditions (i.e. any object exiting the domain from one side re-enters it from the opposite side), then the segmentation process should also include periodic boundaries. This is done by setting the parameter `PeriodicBoundaries` to True.

Note that, in this case, the user must provide the exact size `xmax`, `ymax`, `zmax` of the computational domain (see previous section) : if these parameters are set to the default, automatic computation then no objects will cross the border and the periodic boundaries will have no effect. For this reason, if `PeriodicBoundaries` is True and one of the half-length `xmax`, `ymax` or `zmax` is missing, a warning will be issued to the user. Figure 18 illustrates how the parameters `xmax`, `ymax`, `zmax` and `PeriodicBoundaries` interact.

- **A**. When all the domain's side-lengths are automatically computed, the result of the segmentation is the same whether `PeriodicBoundaries` is True or False, with three clusters identified.
- **B**. With an appropriate user-provided value of `xmax` and automatic computation of `ymax`, two of the previously identified clusters are merged through the periodic x-boundary.
- **C**. With appropriate user-provided values of `xmax` and `ymax`, all previously identified clusters are merged through the periodic boundaries.

The results in this figure can be reproduced by running Script 6 and applying the Paraview macro **Sphereviz** (see section 3.2) to the files thus produced.

Python type is *bool*, default value is False.

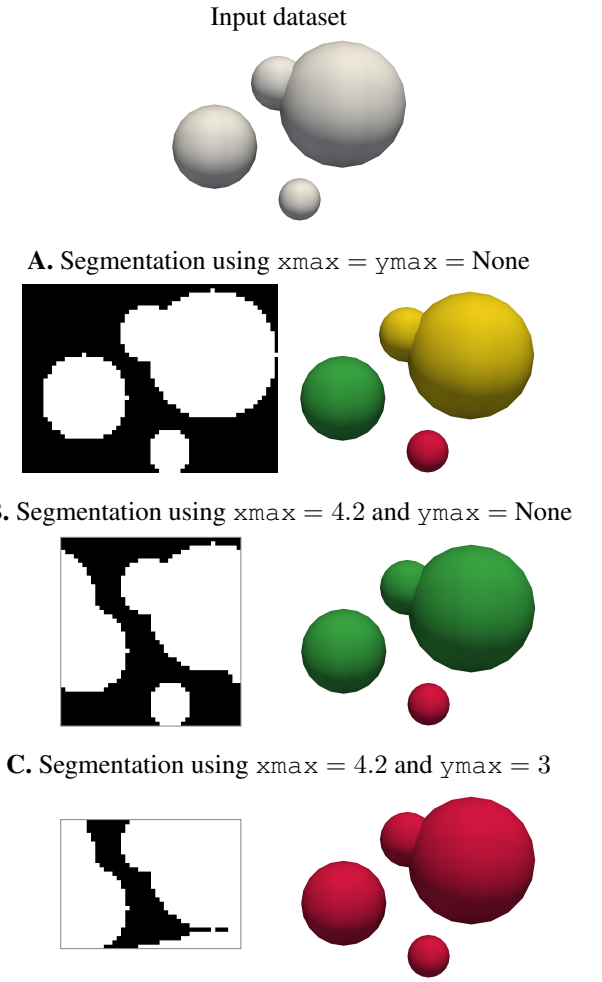


Figure 18: Example of application of periodic boundary conditions to the same data with different domain side-lengths. **Left column :** One slice of the 3D binary image created by the segmentation tool (objects appear in white and background in black). For the sake of clarity, a gray frame has been added to the images in rows **B** and **C**. **Right column :** 3D visualization of the segmented dataset using Paraview (all objects pertaining to the same cluster have the same color).

Python Script 6: Segmentation of a dataset using periodic boundaries.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/domain_size.csv", savefile="domain_size_auto")
WatershedSegmentation("dataset", "demo_data/domain_size.csv", savefile="domain_size_xmax4.2", xmax=4.2,
  ↪ PeriodicBoundaries=True)
WatershedSegmentation("dataset", "demo_data/domain_size.csv", savefile="domain_size_xmax4.2_ymax3",
  ↪ xmax=4.2, ymax=3, PeriodicBoundaries=True)
```

`dil_coeff` : scaling factor

If the `datatype` is "dataset", clustering may be made easier by scaling the size of the spherical objects up or down, that is by multiplying their radius by a factor `dil_coeff` when inserting them in the 3D binary image (see Figure 19 for examples).

For instance, if there are small gaps between objects pertaining to the same cluster, the distance transform will display a local minima at the center of each object and watershed clustering will thus segment the individual objects instead of the cluster (see Figure 19.A). In this case, scaling up the objects by a small factor may fill the holes without unduly connecting separate clusters (see Figure 19.B). This option is especially intended for cases where the user also provides a set of rod-like objects to serve as separators between clusters, as the risk of unwanted cluster merging is then reduced (see Figure 19.C).

If, on the other hand, the user wants to divide a continuous cluster into multiple components based on a notion of objects density, scaling down may allow it by creating holes inside the low density regions (compare Figure 19.D and E). This option is intended for cases where there are large discrepancies in the objects local density.

The results in Figure 19 can be reproduced by running Script 7 and applying the Paraview macro **Sphereviz** (see section 3.2) to the files thus produced.

As a rule, it is best to keep the factor `dil_coeff` close to 1 as values too high or too low are likely to produce unwanted results.

Python type is *float*, default value is 1 (i.e. no scaling).

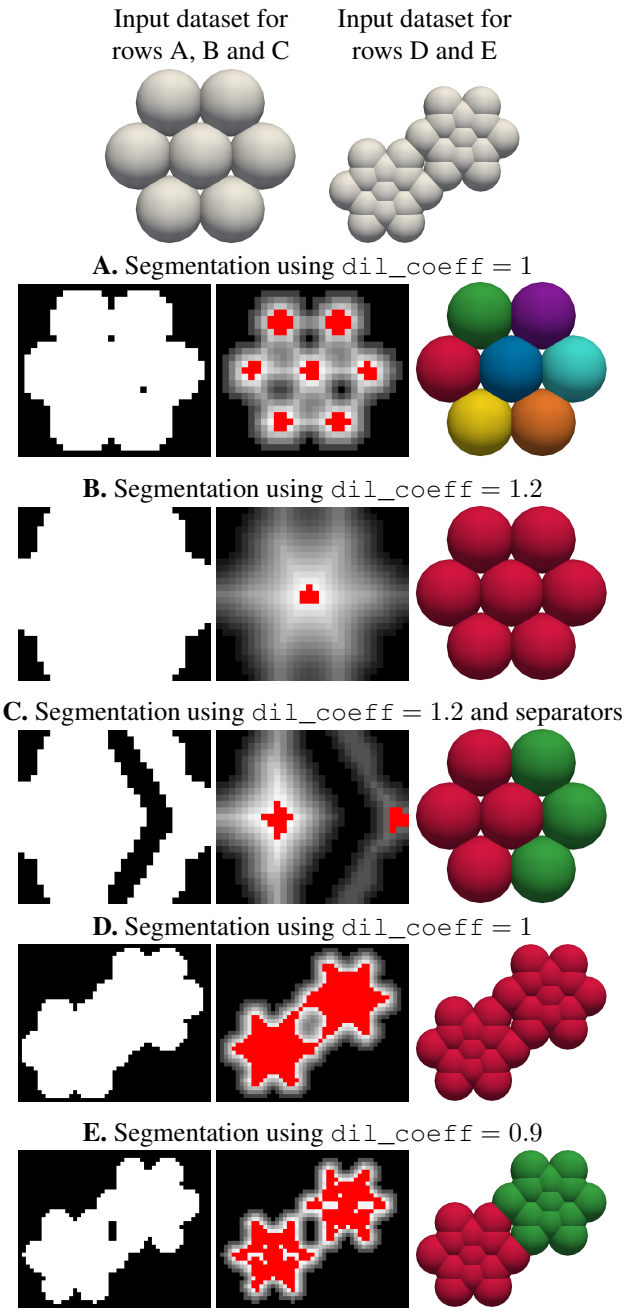


Figure 19: Example of scaling up a dataset. **Left column :** One slice of the binary image created by the segmentation tool (objects appear in white and background in black). **Middle column :** Same slice of corresponding distance map (with local minima indicated in red). **Right column :** 3D visualization of the segmented dataset using Paraview (all objects pertaining to the same cluster have the same color).

Python Script 7: Segmentation of a dataset using different values of the parameter `dil_coeff`.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/scaling_up.csv", savefile="scaling_up1")
WatershedSegmentation("dataset", "demo_data/scaling_up.csv", savefile="scaling_up1.2", dil_coeff=1.2)
WatershedSegmentation("dataset", "demo_data/scaling_up.csv", savefile="scaling_up1.2_withsep",
  ↪  dil_coeff=1.2, InputRods="demo_data/scaling_seps.csv")
WatershedSegmentation("dataset", "demo_data/scaling_down.csv", savefile="scaling_down1")
WatershedSegmentation("dataset", "demo_data/scaling_down.csv", savefile="scaling_down0.9", dil_coeff=0.9)
```

4.3.3 Watershed settings

`pixelsize` : side-lengths of the pixels in the input image

If the `datatype` is "image", the parameter `pixelsize` indicates the relative side-length of the pixels of the image in the x, y and z directions. Unit is implicit and the same than the one of parameter `smooth_coeff` (see next section). This option is intended to account for pixel anisotropy in 3D microscopy images, which usually have a lower resolution in depth (i.e. between two slices) than along the slicing plane, meaning that the pixels have a much larger side-length in the z direction than in the two others.

Taking up the example from section 4.1, considering the pixels to be non-cubic with a depth six times greater than their length and width results in the identification of one more region compared with the initial segmentation. The result of this segmentation is displayed in Figure 20 and can be reproduced by running Script 8.

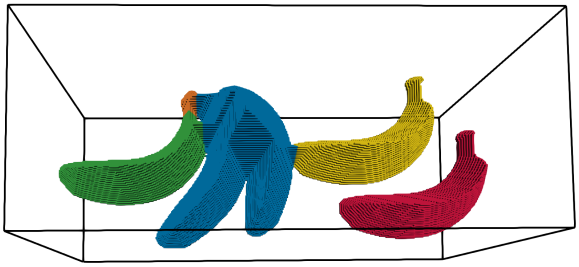


Figure 20: Segmentation of the 3D binary image displayed in Figure 12.A using non-cubic pixels with a depth six times greater than their length and width.

Python type is *list* of three *floats*, default value is $[1.0, 1.0, 1.0]$.

Python Script 8: Segmentation of a dataset tuning the parameter `pixelsize`.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("image", "demo_data/bananas.tiff", pixelsize=[1,1,6])
```

`smooth_coeff` : region smoothing (or peak filtering)

The watershed algorithm segment clusters in a binary image based on seeds. These seeds are usually defined as the local minima in the euclidean distance transform of the image. However, taking all local minima into account is likely to result in over-segmentation, since any indentation in a cluster's surface may generate additional local minima in the distance map. Hence, it is common to filter such shallow minima by applying to the distance transform a smoothing function. This segmentation tool implements a modified version of the h-minima smoothing function, designed to reproduce the behavior of Matlab's imhmin function : it erases all local minima whose depth, with respect to their surroundings, is lesser than `smooth_coeff`.

Figure 21 illustrates the influence of this smoothing step for a 3D image input. In each case, whether the data is over-segmented or not depends on the user's objective.

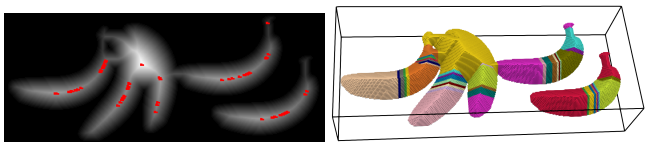
A. The non-smoothed euclidean distance transform presents numerous one-pixel-wide local minima (artificially enlarged on the figure for the sake of visibility), leading to segmentation of 48 different regions.

B. Smoothing the euclidean distance transform by a factor `smooth_coeff` $= 1$ drastically reduces the number of local minima to 4.

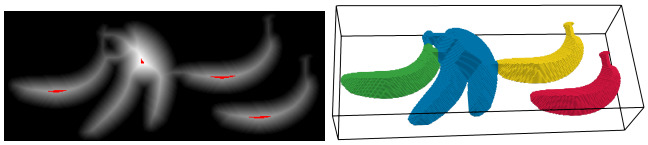
C. Smoothing the euclidean distance transform by a factor `smooth_coeff` $= 10$ reduces the number of local minima to 2, leading to the merging of the three previously distinct but spatially connected regions.

The results in this figure can be reproduced by running Script 9.

A. Segmentation using `smooth_coeff` $= 0$



B. Segmentation using `smooth_coeff` $= 1$



C. Segmentation using `smooth_coeff` $= 10$

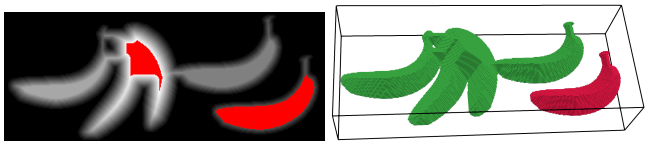


Figure 21: Illustration of the influence of the parameter `smooth_coeff` over the segmentation result. **Left column :** One slice of the distance map (with local minima indicated in red) corresponding to the input dataset. **Right column :** 3D visualization of the segmented image using Paraview.

Python Script 9: Segmentation of a 3D image using different values of the parameter `smooth_coeff`.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("image", "demo_data/bananas.tiff", savefile="bananas_smoothing0", smooth_coeff=0)
WatershedSegmentation("image", "demo_data/bananas.tiff", savefile="bananas_smoothing1")
WatershedSegmentation("image", "demo_data/bananas.tiff", savefile="bananas_smoothing10", smooth_coeff=10)
```

Likewise, Figure 22 illustrates the influence of the parameter `smooth_coeff` over the segmentation of a dataset.

A. The non-smoothed euclidean distance transform presents local minima at the center of each object, leading to segmentation of 13 isolated objects.

B. Smoothing the euclidean distance transform by a factor `smooth_coeff` $= 1$ (px) reduces the number of local minima to 7. Note how the remaining local minima span more pixels due to the smoothing operation.

C. Smoothing the euclidean distance transform by a factor `smooth_coeff` $= 5$ (px) reduces the number of local minima to 1, leading to the segmentation of one cluster containing all the objects.

The results in this figure can be reproduced by running Script 10 and applying the Paraview macro **Sphereviz** (see section 3.2) to the files thus produced.

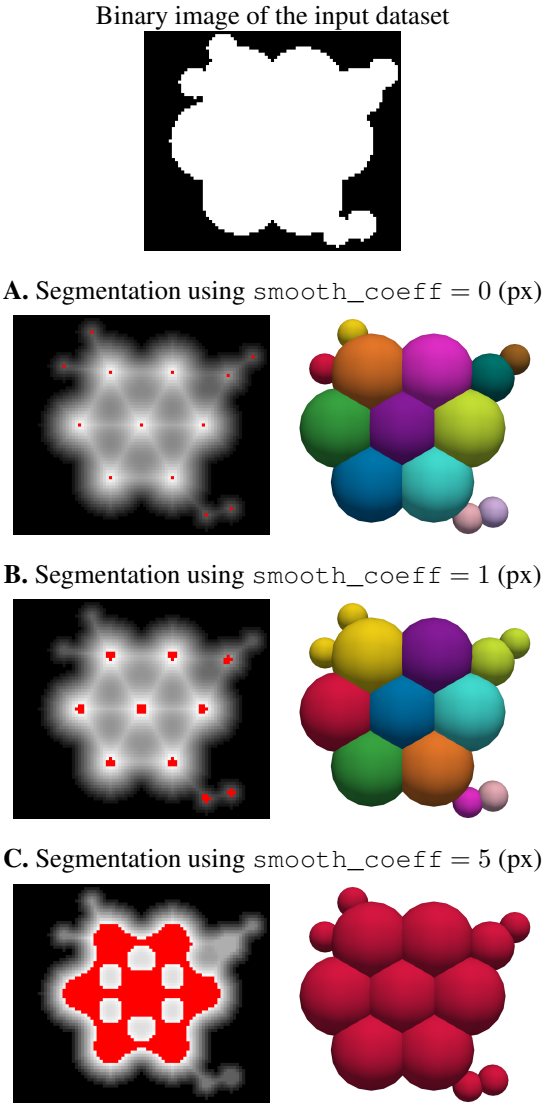


Figure 22: Illustration of the influence of the parameter `smooth_coeff` over the segmentation result. **Left column :** One slice of the distance map (with local minima indicated in red) corresponding to the input dataset. **Right column :** 3D visualization of the segmented dataset using Paraview (all objects pertaining to the same cluster have the same color).

Python Script 10: Segmentation of a dataset tuning the parameter `smooth_coeff`.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/smoothing.csv", savefile="smoothing0", smooth_coeff=0)
WatershedSegmentation("dataset", "demo_data/smoothing.csv", savefile="smoothing1")
WatershedSegmentation("dataset", "demo_data/smoothing.csv", savefile="smoothing5", smooth_coeff=5)
```

Note that the depth `smooth_coeff` is expressed in the same unit than `pixelsize` if `datatype` is "image" (see previous paragraph, by default `pixelsize` is equal to 1, hence a smoothing depth of 1 would equivalent to 1 px); and in (possibly non integral) number of pixels if `datatype` is "dataset". This choice was made because (i) a smoothing of one or two pixels length is usually a good pick to avoid both over and under-segmentation and (ii) for dataset inputs, the user can not choose the pixel size directly (see parameter `resolution` in section 4.3.2 for more details).

Python type is *float*, default value is 1.

4.3.4 Post-processing

`MinSize` : removing the smallest clusters

Small clusters may not interest users. Hence, the parameter `MinSize` allows the user to define a minimum size for a cluster to be considered valid. If the `datatype` is "image", this parameter is expressed in number of pixels in the cluster. If the `datatype` is "dataset", it is expressed in number of objects in the cluster. Appropriate values may vary widely depending on the user's purpose.

Please note that this option does not affect the segmentation process in itself : it is only a post-processing step which erase small clusters. It is mainly intended for users who plan to do statistical computation on the segmented clusters and do not want to take into account small, miscellaneous clusters. It *does not* merge multiple small clusters into a large cluster or blend a small cluster into a neighboring large cluster (such results could however be achieved using the parameter `smooth_coeff`, see corresponding section above).

If the `datatype` is "image", invalid clusters will be removed from the segmented image (i.e. blended into the background) and the remaining clusters will be numbered continuously starting from 1 (with 0 as the background value). If the `datatype` is "dataset", objects pertaining to invalid clusters will get the cluster index -1, while valid clusters are numbered continuously starting from 0 to be consistent with Python indexing.

Python type is *int*, default value is 1 (meaning that all clusters will be valid).

5 Practical examples

The following section present some realistic examples of use of the SegViz tool, allowing to reproduce the graphical abstract on the front page of this document.

5.1 Visualizing a binary tiff file

To reproduce the top-left image on the front page of this document, first unzip the file `demo_data/mouse_adipose_tissue.bz2` and open the produced file with Paraview. If a popup window appears, select the "TIFF serie reader".

This image of a mouse subcutaneous adipose tissue is a courtesy of the Restore Institute. It was acquired with a light-sheet microscope "Light sheet Z7" from Zeiss and have anisotropic pixels of size $18.4\mu m \times 18.4\mu m \times 10\mu m$. To match this in Paraview, tick the "Use Custom Data Spacing" checkbox in the "Properties" panel and fill in the three boxes just below with respectively 1.84, 1.84 and 1, then click on the "Apply" button. At this step, nothing should appear yet in the display panel.

Select the "Volume" setting in the "Representation" drop-down menu to make the data visible. The coloring setting will automatically change from "Solid Color" to "Tiff Scalars". Click on the "Edit" button below to open the Color Map Editor, then click on the "Choose preset" button in this editor. In the popup window that now appears, select the X Ray colormap. Click on the "Apply" button and close the popup. It may be necessary to also click on the "Render Views" button at the very bottom of the Color Map Editor to update the display.

5.2 Segmenting a binary tiff file

To reproduce the bottom-left image on the front page of this document, first segment the content of the file `demo_data/mouse_adipose_tissue.bz2` (after unzipping it) using the python script below. Alternatively, you can skip the segmentation step and use the content of the file `demo_data/mouse_adipose_tissue_seg.bz2` (after unzipping it).

```
from SegTool import WatershedSegmentation
WatershedSegmentation("image", "demo_data/mouse_adipose_tissue.tif", pixelsize=[1.84, 1.84, 1],
  ↪  smooth_coeff=10, MinSize=400000)
```

Let's discuss this script. As indicated before, the image to segment have anisotropic pixels of aspect ratio $1.84 : 1.84 : 1$ which should be provided to the `pixelsize` option (using an implicit unit of length $L = 10\mu m$). The surface of the data shows some irregularities whose impact on the segmentation can be eliminated by setting the smoothing coefficient to $10L = 100\mu m$ (that is between 5 and 10 pixel-width depending on the direction). Lastly, the data contains a lot of small, isolated spots that can be removed from the segmented map by setting the minimum cluster size to 400000 pixels.

Open the segmented image with Paraview using exactly the same protocol as in the previous section, except for the choice of the colormap which must be Rainbow Desaturated.

5.3 Visualizing datasets

To reproduce the top-right image on the front page of this document, open the files `demo_data/dataset_spheres.csv` and `demo_data/dataset_rods.csv` with Paraview, then apply the macro **Sphereviz.py** to the former and the macro **Rodviz.py** to the latter (see section 3.2 for the detailed process).

To add the black cubic frame, open the "Sources" drop-down menu in Paraview menu bar and select the entry "Geometric Shapes > Box". In the "Properties" panel, set the value of "X Length", "Y Length" and "Z Length" to 30, then click on the

"Apply" button. A full light-gray cube should now be visible in the display panel. In the "Representation" drop-down menu that just appeared, select the "Outline" setting. In the Styling section, set the "Line Width" value to 6. Lastly, click on the "Edit" button in the Coloring section and select black.

5.4 Segmenting a dataset

To reproduce the bottom-right image on the front page of this document, first segment the dataset `demo_data/dataset_spheres.csv` using the python script below. Alternatively, you can skip the segmentation step and use the file `demo_data/dataset_spheres_seg.csv`.

```
from SegTool import WatershedSegmentation
WatershedSegmentation("dataset", "demo_data/dataset_spheres.csv", InputRods="demo_data/dataset_rods.csv",
  ↪  PeriodicBoundaries=True, xmax=15, ymax=15, zmax=15, dil_coeff=1.5, resolution=3, MinSize=10)
```

Let's discuss this script. The dataset to segment comes from a numerical simulation of a system mixing spherical and rod-like objects, so the corresponding dataset of rod-like objects (stored in the file `demo_data/dataset_rods.csv`) can be used to help separate the clusters of spheres. The simulation was conducted in a cubic domain of half-length 15 with periodic boundary conditions in all directions, which must be accounted for via the `PeriodicBoundaries`, `xmax`, `ymax` and `zmax` options. Due to some of the simulation settings, the spherical objects are gathered in somewhat loose clusters with a lot of interstitial space. Thanks to the presence of rod-like objects between the clusters, we can fill the interstitial space by scaling up the objects size by a factor 1.5 without too much risk of unduly fusing together different clusters. The resolution can be kept low to reduce computational costs. Finally, the dataset contains 573 objects, so a minimal cluster size of 10 objects seems reasonable if we are only interested in the main, big structures.

Open the segmented file with Paraview using exactly the same protocol as in the previous section. Use the file `demo_data/dataset_rods_colored.csv`, which contains a "color" column, to obtain colored rod-like objects.

Going further, you can switch to translated coordinates to better visualize the shape of the sphere clusters that span through one or more of the (periodic) borders of the spacial domain. To do this, first click on the "Spheres DataTable" item in the "Pipeline Browser" panel. Then, in the "Properties" panel, change the value of the "X Column", "Y Column" and "Z Column" properties respectively from X, Y and Z to X_cluster, Y_cluster and Z_cluster. The result is displayed in Figure 23.

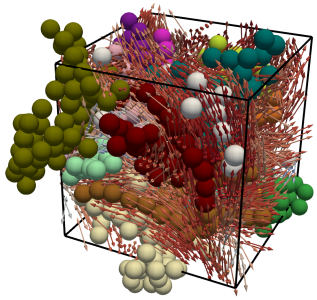


Figure 23: Example of use of the translated coordinates when visualizing in Paraview a dataset segmented using periodic boundary conditions. Compare with the bottom-right panel on the front page of this document, which uses the same data and natural coordinates.

Acknowledgements and funding

Restore Institute for the biological data ? M. Vigneau, L. Pieruccioni

The authors thank Charlotte Brunet, Juyeon Kim and Marion Saint-Pée, students of the L3 CMI Mécanique of Sorbonne University, who developed the first version of the visualization pipeline during their internship at ISIR.

This project was supported by Agence Nationale de la Recherche (ANR) through the ENERGENCE project (under grant number ANR-22-CE45-0024-01) and the CARe graduate school (under grant number ANR-18-EURE-0003), and by Sorbonne Alliance University with the Emergence project MATHREGEN (under grant number S29-05Z101).

Bibliography

[1] P. Chassonnery, J. Paupert, A. Lorsignol, C. Sévérac, M. Ousset, P. Degond, L. Casteilla and D. Peurichard. "Fiber crosslinking drives the emergence of order in a three-dimensional dynamical network model." *Royal Society Open Science* **11**, 231456 (2024). doi: 10.1098/rsos.231456

[2] P. Chassonnery, D. Peurichard and S. Haliyo. "3D visualisation and segmentation tools for systems of spherical and rod-like objects." *(in preparation)*

[3] P. Chassonnery, J. Paupert, A. Lorsignol, C. Sévérac, M. Ousset, L. Casteilla and D. Peurichard. "Emergence and robustness of 3D complex tissue architectures from simple physical forces." *(in preparation)*

[4] J. Ahrens, B. Geveci and C. Law. "ParaView: An End-User Tool for Large Data Visualization" in *Visualization Handbook*, C. D. Hansen, C. R. Johnson, Eds. (Elsevier, 2005), pp. 717–731. doi: 10.1016/B978-012387582-2/50038-1

[5] S. Trubetskoy. "List of 20 Simple, Distinct Colors." https://sashamaps.net/docs/resources/20-colors/ Accessed: 2024-04-17

[6] F. Meyer. "Topographic distance and watershed lines." *Signal Processing* **38**, 113–125 (1994). doi: 10.1016/0165-1684(94)90060-4

[7] page de ref python sur l'usage du watershed ?

[8] ref octave (version open-source de matlab) pour la fonction imhmin ?

Appendix : brief description of the segmentation algorithm

The function **WatershedSegmentation** takes as input a parameter describing the type of data to treat (3D binary image or dataset of spherical objects), the data in question and a number of optional parameters (see chapter 4 above for more details).

For both types of input we apply the same algorithm, implemented in the function **MapRegionsUsingWatershed**. This algorithm segments a 3D binary image into separate regions based on spatial proximity, using the watershed transform, and can be broken down as follows (the name of the related sub-functions are given in bold text) :

1. Apply euclidean distance transform to create a grayscale map indicating, for each white pixel (foreground) of the initial binary image, its distance to the closest black pixel (background).

   ↳ **scipy.ndimage.distance_transform_edt**

2. Filter this distance map to remove shallow local minima.

   ↳ **imhmin**

3. Identify and label local minima in the filtered distance map.

   ↳ **skimage.morphology.local_maxima**

   ↳ **scipy.ndimage.label**

4. Apply watershed algorithm to the filtered distance map, using local minima as seeds.

   ↳ **skimage.segmentation.watershed**

If the input data is a 3D binary image, no further processing is required; the segmentation map will be saved into a .tiff file as a labeled 3D image.

If it is a dataset instead, then it must be pre-processed into a 3D binary image where objects are white and background is black (using the function **Create3Dbinaryimage**), and the segmentation map must be post-processed to identify the cluster index of each object of the dataset (using the function **ClusterizeFromMap**). The result of this clusterization will be saved into a .csv file. Optionally, the clusterization map can also be saved in a .tiff file as a 3D labeled image.